

One user, known as the *superuser*, has special privileges within the system. The superuser account has user ID 0, and normally has the login name *root*. On typical UNIX systems, the superuser bypasses all permission checks in the system. Thus, for example, the superuser can access any file in the system, regardless of the permissions on that file, and can send signals to any user process in the system. The system administrator uses the superuser account to perform various administrative tasks on the system.

2.4 Single Directory Hierarchy, Directories, Links, and Files

The kernel maintains a single hierarchical directory structure to organize all files in the system. (This contrasts with operating systems such as Microsoft Windows, where each disk device has its own directory hierarchy.) At the base of this hierarchy is the *root directory*, named */* (slash). All files and directories are children or further removed descendants of the root directory. Figure 2-1 shows an example of this hierarchical file structure.

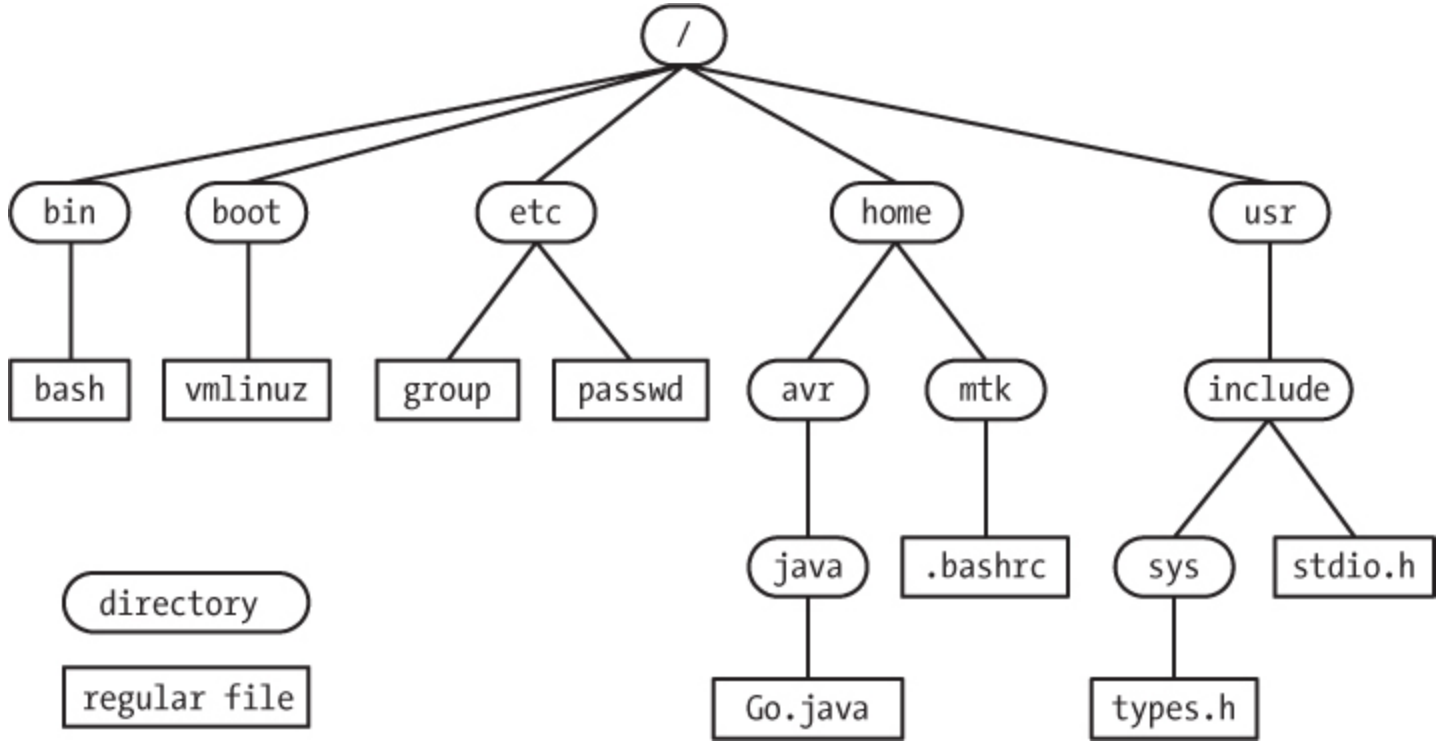


Figure 2-1: Subset of the Linux single directory hierarchy

File types

Within the file system, each file is marked with a *type*, indicating what kind of file it is. One of these file types denotes ordinary data files, which are usually called *regular* or *plain* files to distinguish them from other file types. These other file types include devices, pipes, sockets, directories, and symbolic links.

The term *file* is commonly used to denote a file of any type, not just a regular file.

Directories and links

A *directory* is a special file whose contents take the form of a table of filenames coupled with references to the corresponding files. This filename-plus-reference association is called a *link*, and files may have multiple links, and thus multiple names, in the same or in different directories.

Directories may contain links both to files and to other directories. The links between directories establish the directory hierarchy shown in Figure 2-1.

Every directory contains at least two entries: *.* (dot), which is a link to the directory itself, and *..* (dot-dot), which is a link to its *parent directory*, the directory above it in the hierarchy. Every directory, except the root directory, has a parent. For the root directory, the dot-dot entry is a link to the root directory itself (thus, */..* equates to */*).

Symbolic links

Like a normal link, a *symbolic link* provides an alternative name for a file. But whereas a normal link is a filename-plus-pointer entry in a directory list, a symbolic link is a specially marked file containing the name of another file. (In other words, a symbolic link has a filename-plus-pointer entry in a directory, and the file referred to by the pointer contains a string that names another file.) This latter file is often called the *target* of the symbolic link, and it is common to say that the symbolic link “points” or “refers” to the target file. When a pathname is specified in a system call, in most circumstances, the kernel automatically *dereferences* (or synonymously, *follows*) each symbolic link in the pathname, replacing it with the filename to which it points. This process may happen recursively if the target of a symbolic link is itself a symbolic link. (The kernel imposes limits on the number of dereferences to handle the possibility of circular chains of symbolic links.) If a symbolic link refers to a file that doesn’t exist, it is said to be a *dangling link*.

Often *hard link* and *soft link* are used as alternative terms for normal and symbolic links. The reasons for having two different types of links are explained in Chapter 18.

Filenames

On most Linux file systems, filenames can be up to 255 characters long. Filenames may contain any characters except slashes (*/*) and null characters (*\0*). However, it is advisable to employ only letters and digits, and the *.* (period), *_* (underscore), and *-* (hyphen) characters. This 65-character set, [*-._a-zA-Z0-9*], is referred to in SUSv3 as the *portable filename character set*.

We should avoid the use of characters in filenames that are not in the portable filename character set because those characters may have special meanings within the shell, within regular expressions, or in other contexts. If a filename containing characters with special meanings appears in such contexts, then these characters must be *escaped*; that is, specially marked—typically with a preceding backslash (**)—to indicate that they should not be interpreted with those special meanings. In contexts where no escape mechanism is available, the filename is not usable.

We should also avoid filenames beginning with a hyphen (*-*), since such filenames may be mistaken for options when specified in a shell command.

Pathnames

A *pathname* is a string consisting of an optional initial slash (*/*) followed by a series of filenames separated by slashes. All but the last of these component filenames identifies a directory (or a symbolic link that resolves to a directory). The last component of a pathname may identify any type of file, including a directory. The series of component filenames preceding the final slash is sometimes referred to as the *directory* part of a pathname, while the name following the final slash is sometimes referred to as the file or *base* part of the pathname.

A pathname is read from left to right; each filename resides in the directory specified by the preceding part of the pathname. The string *..* can be used anywhere in a pathname to refer to the parent of the location so far specified in the pathname.

A pathname describes the location of a file within the single directory hierarchy, and is either absolute or relative:

- An *absolute pathname* begins with a slash (*/*) and specifies the location of a file with respect to the root directory. Examples of absolute pathnames for files in Figure 2-1 are */home/mtk/.bashrc*, */usr/include*, and */* (the pathname of the root directory).
- A *relative pathname* specifies the location of a file relative to a process’s current working directory (see below), and is distinguished from an absolute pathname by the absence of an initial slash. In Figure 2-1, from the directory *usr*, the file *types.h* could be referenced using the relative pathname *include/sys/types.h*, while from the directory *avr*, the file *.bashrc* could be accessed using the relative pathname *../mtk/.bashrc*.

Current working directory

Each process has a *current working directory* (sometimes just referred to as the process’s *working directory* or *current directory*). This is the process’s “current location” within the single directory hierarchy, and it is from this directory that relative pathnames are interpreted for the process.

A process inherits its current working directory from its parent process. A login shell has its initial current working directory set to the location named in the home directory field of the user’s password file entry. The shell’s current working directory can be changed with the *cd* command.

File ownership and permissions

Each file has an associated user ID and group ID that define the owner of the file and the group to which it belongs. The ownership of a file is used to determine the access rights available to users of the file.

For the purpose of accessing a file, the system divides users into three categories: the *owner* of the file (sometimes termed the *user* of the file), users who are members of the group matching the file’s group ID (*group*), and the rest of the world (*other*). Three permission bits may be set for each of these categories of user (making a total of nine permission bits): *read* permission allows the contents of the file to be read; *write* permission allows modification of the contents of the file; and *execute* permission allows execution of the file, which is either a program or a script to be processed by some interpreter (usually, but not always, one of the shells).

These permissions may also be set on directories, although their meanings are slightly different: *read* permission allows the contents of (i.e., the filenames in) the directory to be listed; *write* permission allows the contents of the directory to be changed (i.e., filenames can be added, removed, and changed); and *execute* (sometimes called *search*) permission allows access to files within the directory (subject to the permissions on the files themselves).

2.5 File I/O Model

One of the distinguishing features of the I/O model on UNIX systems is the concept of *universality of I/O*. This means that the same system calls (*open()*, *read()*, *write()*, *close()*, and so on) are used to perform I/O on all types of files, including devices. (The kernel translates the application’s I/O requests into appropriate file-system or device-driver operations that perform I/O on the target file or device.) Thus, a program employing these system calls will work on any type of file.

The kernel essentially provides one file type: a sequential stream of bytes, which, in the case of disk files, disks, and tape devices, can be randomly accessed using the *lseek()* system call.

Many applications and libraries interpret the *newline* character (ASCII code 10 decimal, sometimes also known as *linefeed*) as terminating one line of text and commencing another. UNIX systems have no *end-of-file* character; the end of a file is detected by a read that returns no data.

File descriptors

The I/O system calls refer to open files using a *file descriptor*, a (usually small) nonnegative integer. A file descriptor is typically obtained by a call to *open()*, which takes a pathname argument specifying a file upon which I/O is to be performed.

Normally, a process inherits three open file descriptors when it is started by the shell: descriptor 0 is *standard input*, the file from which the process takes its input; descriptor 1 is *standard output*, the file to which the process writes its output; and descriptor 2 is *standard error*, the file to which the process writes error messages and notification of exceptional or abnormal conditions. In an interactive shell or program, these three descriptors are normally connected to the terminal. In the *stdio* library, these descriptors correspond to the file streams *stdin*, *stdout*, and *stderr*.

The *stdio* library

To perform file I/O, C programs typically employ I/O functions contained in the standard C library. This set of functions, referred to as the *stdio* library, includes *fopen()*, *fclose()*, *scanf()*, *printf()*, *fgetc()*, *fputs()*, and so on. The *stdio* functions are layered on top of the I/O system calls (*open()*, *close()*, *read()*, *write()*, and so on).

We assume that the reader is already familiar with the C standard I/O (*stdio*) functions, and don’t cover them in this book. Further information on the *stdio* library can be found in [Kernighan & Ritchie, 1988], [Harbison & Steele, 2002], [Plauger, 1992], and [Stevens & Rago, 2005].

4

FILE I/O: THE UNIVERSAL I/O MODEL

We now start to look in earnest at the system call API. Files are a good place to start, since they are central to the UNIX philosophy. The focus of this chapter is the system calls used for performing file input and output. We introduce the concept of a file descriptor, and then look at the system calls that constitute the so-called universal I/O model. These are the system calls that open and close a file, and read and write data. We focus on I/O on disk files. However, much of the material covered here is relevant for later chapters, since the same system calls are used for performing I/O on all types of files, such as pipes and terminals. Chapter 5 extends the discussion in this chapter with further details on file I/O. One other aspect of file I/O, buffering, is complex enough to deserve its own chapter. Chapter 13 covers I/O buffering in the kernel and in the *stdio* library.

4.1 Overview

All system calls for performing I/O refer to open files using a *file descriptor*, a (usually small) nonnegative integer. File descriptors are used to refer to all types of open files, including pipes, FIFOs, sockets, terminals, devices, and regular files. Each process has its own set of file descriptors. By convention, most programs expect to be able to use the three standard file descriptors listed in Table 4-1. These three descriptors are opened on the program's behalf by the shell, before the program is started. Or, more precisely, the program inherits copies of the shell's file descriptors, and the shell normally operates with these three file descriptors always open. (In an interactive shell, these three file descriptors normally refer to the terminal under which the shell is running.) If I/O redirections are specified on a command line, then the shell ensures that the file descriptors are suitably modified before starting the program.

Table 4-1: Standard file descriptors

File descriptor	Purpose	POSIX name	<i>stdio</i> stream
0	standard input	STDIN_FILENO	<i>stdin</i>
1	standard output	STDOUT_FILENO	<i>stdout</i>
2	standard error	STDERR_FILENO	<i>stderr</i>

When referring to these file descriptors in a program, we can use either the numbers (0, 1, or 2) or, preferably, the POSIX standard names defined in `<unistd.h>`.

Although the variables *stdin*, *stdout*, and *stderr* initially refer to the process's standard input, output, and error, they can be changed to refer to any file by using the *freopen()* library function. As part of its operation, *freopen()* may change the file descriptor underlying the reopened stream. In other words, after an *freopen()* on *stdout*, for example, it is no longer safe to assume that the underlying file descriptor is still 1.

The following are the four key system calls for performing file I/O (programming languages and software packages typically employ these calls only indirectly, via I/O libraries):

- fd* = *open(pathname, flags, mode)* opens the file identified by *pathname*, returning a file descriptor used to refer to the open file in subsequent calls. If the file doesn't exist, *open()* may create it, depending on the settings of the *flags* bit-mask argument. The *flags* argument also specifies whether the file is to be opened for reading, writing, or both. The *mode* argument specifies the permissions to be placed on the file if it is created by this call. If the *open()* call is not being used to create a file, this argument is ignored and can be omitted.
- numread* = *read(fd, buffer, count)* reads at most *count* bytes from the open file referred to by *fd* and stores them in *buffer*. The *read()* call returns the number of bytes actually read. If no further bytes could be read (i.e., end-of-file was encountered), *read()* returns 0.
- numwritten* = *write(fd, buffer, count)* writes up to *count* bytes from *buffer* to the open file referred to by *fd*. The *write()* call returns the number of bytes actually written, which may be less than *count*.
- status* = *close(fd)* is called after all I/O has been completed, in order to release the file descriptor *fd* and its associated kernel resources.

Before we launch into the details of these system calls, we provide a short demonstration of their use in Listing 4-1. This program is a simple version of the *cp(1)* command. It copies the contents of the existing file named in its first command-line argument to the new file named in its second command-line argument.

We can use the program in Listing 4-1 as follows:

```
$ ./copy oldfile newfile
```

Listing 4-1: Using I/O system calls

```
fileio/copy.c

#include <sys/stat.h>
#include <fcntl.h>
#include "tapi_hdr.h"

#ifndef BUF_SIZE
#define BUF_SIZE 1024
#endif

int
main(int argc, char *argv[])
{
    int inputFd, outputFd, openFlags;
    mode_t filePerms;
    ssize_t numRead;
    char buf[BUF_SIZE];

    if (argc != 3 || strcmp(argv[1], "--help") == 0)
        usageErr("%s old-file new-file\n", argv[0]);

    /* Open input and output files */

    inputFd = open(argv[1], O_RDONLY);
    if (inputFd == -1)
        errExit("opening file %s", argv[1]);

    openFlags = O_CREAT | O_WRONLY | O_TRUNC;
    filePerms = S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP |
                S_IROTH | S_IWOTH; /* rw-rw-rw- */
    outputFd = open(argv[2], openFlags, filePerms);
    if (outputFd == -1)
        errExit("opening file %s", argv[2]);

    /* Transfer data until we encounter end of input or an error */

    while ((numRead = read(inputFd, buf, BUF_SIZE)) > 0)
        if (write(outputFd, buf, numRead) != numRead)
            fatal("write() returned error or partial write occurred");
    if (numRead == -1)
        errExit("read");

    if (close(inputFd) == -1)
        errExit("close input");
    if (close(outputFd) == -1)
        errExit("close output");

    exit(EXIT_SUCCESS);
}
```

4.2 Universality of I/O

One of the distinguishing features of the UNIX I/O model is the concept of *universality of I/O*. This means that the same four system calls—*open()*, *read()*, *write()*, and *close()*—are used to perform I/O on all types of files, including devices such as terminals. Consequently, if we write a program using only these system calls, that program will work on any type of file. For example, the following are all valid uses of the program in Listing 4-1:

```
$ ./copy test test.old      Copy a regular file
$ ./copy a.txt /dev/tty     Copy a regular file to this terminal
$ ./copy /dev/tty b.txt     Copy input from this terminal to a regular file
$ ./copy /dev/pts/16 /dev/tty Copy input from another terminal
```

Universality of I/O is achieved by ensuring that each file system and device driver implements the same set of I/O system calls. Because details specific to the file system or device are handled within the kernel, we can generally ignore device-specific factors when writing application programs. When access to specific features of a file system or device is required, a program can use the catchall *ioctl()* system call (Section 4.8), which provides an interface to features that fall outside the universal I/O model.

4.3 Opening a File: open()

The *open()* system call either opens an existing file or creates and opens a new file.

#include <sys/stat.h>
#include <fcntl.h>

int open(const char *pathname, int flags, ... /* mode_t mode */);

Returns file descriptor on success, or -1 on error

The file to be opened is identified by the *pathname* argument. If *pathname* is a symbolic link, it is dereferenced. On success, *open()* returns a file descriptor that is used to refer to the file in subsequent system calls. If an error occurs, *open()* returns -1 and *errno* is set accordingly.

The *flags* argument is a bit mask that specifies the *access mode* for the file, using one of the constants shown in Table 4-2.

Early UNIX implementations used the numbers 0, 1, and 2 instead of the names shown in Table 4-2. Most modern UNIX implementations define these constants to have those values. Thus, we can see that `O_RDWR` is not equivalent to `O_RDONLY | O_WRONLY`; the latter combination is a logical error.

When *open()* is used to create a new file, the *mode* bit-mask argument specifies the permissions to be placed on the file. (The *mode_t* data type used to type *mode* is an integer type specified in SUSv3.) If the *open()* call doesn't specify `O_CREAT`, *mode* can be omitted.

Table 4-2: File access modes

Access mode	Description
O_RDONLY	Open the file for reading only
O_WRONLY	Open the file for writing only
O_RDWR	Open the file for both reading and writing